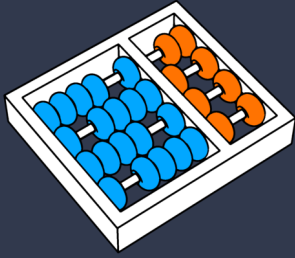




UNICAMP



Construções de pipelines com Langchain

Prof. Dr. André Gomes Regino

regino@unicamp.br

Agenda

01

Introdução aos Pipelines

O que é um pipeline e por que ele é necessário.

02

Etapas Típicas

Pré-processamento, geração e pós-processamento.

03

Chains & Fluxo

Conceitos de Chains, controle de fluxo e roteamento.

04

Confiabilidade

Tratamento de erros, logging e debugging.

05

Boas Práticas

Modularização e reutilização estratégica de componentes.

Objetivos

Objetivos

- **Descrever** as etapas de um pipeline completo com LLM
- **Construir** pipelines sequenciais e com roteamento usando LCEL
- **Aplicar** tratamento de erros, retries e fallbacks em pipelines reais
- **Depurar** pipelines com verbose, callbacks e LangSmith

O que é um pipeline?

Abordagem Ingênua: Ponto Isolado

```
# Chamada única e direta
resposta = llm.invoke("Resuma este texto: " + texto)
```

Abordagem Robusta: Pipeline Multietapas

```
# Etapas com responsabilidades claras
texto_limpo = pre_processar(texto)
contexto = recuperar_contexto(texto_limpo)
resposta_raw = gerar_resposta(contexto)
saida_final = pos_processar(resposta_raw)
```

- **Modularidade:** Cada etapa faz apenas uma coisa bem feita.
- **Previsibilidade:** Facilita a depuração e testes de cada unidade.
- **Escalabilidade:** Permite adicionar roteamento e tratamento de erros.
- **Enriquecimento:** Garante que a LLM receba o melhor contexto possível.

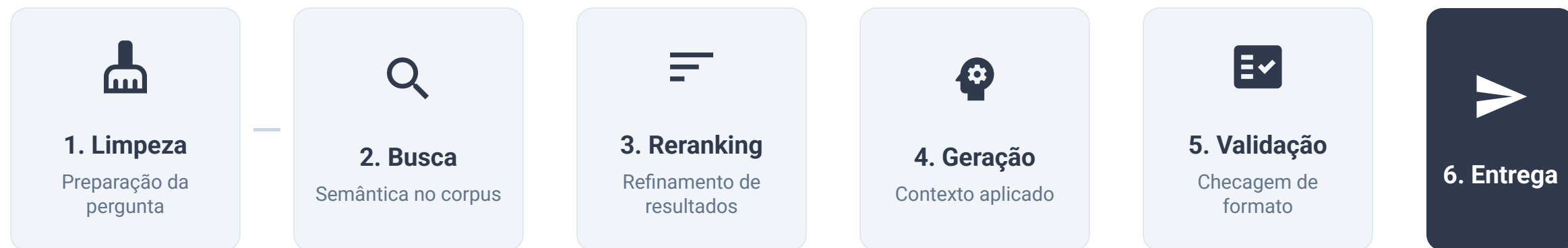
Por que Pipelines são Necessárias

Por que uma única chamada ao LLM não resolve tarefas complexas?

Limitação do LLM isolado	Solução via pipeline
Contexto de tokens finito	Recuperar só o trecho relevante (RAG)
Não valida o próprio output	Etapa de pós-processamento com parser
Não sabe quando a resposta está errada	Retry + validação automática
Não acessa dados externos	Tool calling + etapa de busca
Output não estruturado	Output parser antes de entregar

Por que Pipelines são Necessárias

Exemplo real: um sistema de Q&A sobre documentos jurídicos exige 6 etapas:



Cada uma das etapas possui uma **responsabilidade distinta** garantindo robustez ao sistema.

Exemplo de Pipeline com LLM

Input: "qual é o prazo de entrega?" ← string bruta do usuário

1. Pré-processamento

- lowercase
- validação de comprimento
- detectar idioma

2. Recuperação

- embedding da pergunta
- busca semântica
- top-3 chunks relevantes

3. Geração (LLM)

- montar prompt com contexto
- chamar gpt-5

4. Pós-processamento

- *StrOutputParser*
- validar resposta
- logar latência/tokens

Output: "O prazo de entrega é de 5 dias úteis." ← resposta final para o usuário

Etapas Típicas

As 4 etapas típicas de um pipeline com LLM:

Etapa 01

Pré-processamento

Limpa, normaliza e valida a entrada

COMPONENTES

- RunnableLambda
- RunnablePassthrough

Etapa 02

Recuperação de contexto

Busca informação relevante no corpus

COMPONENTES

- VectorStore.as_retriever()
- RunnableParallel

Etapa 03

Geração

Chama o LLM com prompt estruturado

COMPONENTES

- ChatPromptTemplate
- ChatOpenAI

Etapa 04

Pós-processamento

Formata, valida e estrutura a saída

COMPONENTES

- StrOutputParser
- JsonOutputParser
- PydanticOutputParser

Pré-processamento: RunnableLambda

REQUISITOS

Versão: langchain-core >= 0.3

Transformação funcional simplificada.

Vantagens do RunnableLambda

- **Integração nativa:** Qualquer função Python vira um componente do pipeline.
- **Interface unificada:** Mantém o método `invoke()` padrão.
- **Observabilidade:** Pode ser debugado, logado e testado isoladamente.

```
1 from langchain_core.runnables import RunnableLambda
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_openai import ChatOpenAI
4
5 def pre_processar(entrada: dict) -> dict:
6     """Limpa e valida a entrada antes de passar para o LLM."""
7     texto = entrada.get("pergunta", "")
8
9     # Limpeza
10    texto = texto.strip().lower()
11
12    # Validação
13    if len(texto) < 3:
14        raise ValueError(f"Entrada muito curta: '{texto}'")
15    if len(texto) > 1000:
16        texto = texto[:1000] # truncar
17
18    return {"pergunta": texto}
19
20 # Encadeando como Runnable
21 etapa_pre = RunnableLambda(pre_processar)
22
23 prompt = ChatPromptTemplate.from_messages([
24     ("system", "Você é um assistente útil."),
25     ("human", "{pergunta}")
26 ])
27 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
28
29 # Pipeline com pré-processamento
30 pipeline = etapa_pre | prompt | llm
31 resultado = pipeline.invoke({"pergunta": " O QUE É LANGCHAIN? "})
```

Geração com LLM

REQUISITOS

Versão: langchain-core >= 0.3

langchain-openai >= 0.2

Boas Práticas de Geração

No **system prompt**, sempre defina:

- **Tom de voz:** Garante consistência.
- **Limites:** Define o escopo da resposta.
- **Fallback:** Use ("não sei") para evitar alucinações.

👉 Reduz erros e torna o output mais previsível.

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_openai import ChatOpenAI
3 from langchain_core.output_parsers import StrOutputParser
4
5 # Prompt estruturado – system define comportamento, humano traz o dado
6 prompt = ChatPromptTemplate.from_messages([
7     ("system", """Você é um especialista em {dominio}.
8 Responda de forma concisa, em até 3 frases.
9 Se não souber, diga 'Não tenho informação suficiente.'"""),
10    ("human", "Contexto:\n{contexto}\n\nPergunta: {pergunta}")
11 ])
12
13 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
14
15 # Chain de geração isolada – testável independentemente
16 chain_geracao = prompt | llm | StrOutputParser()
17
18 # Teste unitário da etapa de geração
19 resultado = chain_geracao.invoke({
20     "dominio": "direito trabalhista",
21     "contexto": "O aviso prévio é de 30 dias para contratos...",
22     "pergunta": "Qual o prazo de aviso prévio?"
23 })
24 print(resultado)
```

Pós-Processamento

REQUISITOS

Versão: langchain-core >= 0.3

Output Parsers

Transformando texto em dado estruturado.

Conceito Chave:

Pós-processamento é o contrato de saída. Definir o parser é garantir o que o sistema entrega.



```
1 from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
2 from langchain_core.pydantic_v1 import BaseModel, Field
3 from langchain_core.output_parsers import PydanticOutputParser
4
5 # 1. StrOutputParser – extrai só o texto da AIMessage
6 parser_str = StrOutputParser()
7
8 # 2. JsonOutputParser – força saída JSON
9 parser_json = JsonOutputParser()
10
11 # 3. PydanticOutputParser – saída tipada com validação
12 class RespostaTicket(BaseModel):
13     categoria: str = Field(description="Categoria do ticket: técnico, comercial, outro")
14     prioridade: int = Field(description="Prioridade de 1 (baixa) a 5 (crítica)")
15     resumo: str = Field(description="Resumo em até 20 palavras")
16
17 parser_pydantic = PydanticOutputParser(pydantic_object=RespostaTicket)
```

Pós-Processamento

TABELA DE COMPARAÇÃO DE SAÍDAS

Parser	Saída	Quando usar
StrOutputParser	str	Respostas em texto livre
JsonOutputParser	dict	Integração com APIs/sistemas
PydanticOutputParser	objeto tipado	Validação rigorosa de schema

Chains

Chain simples com LCEL

```
# Importações necessárias
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Isso substitui o LLMChain legado
chain = (
    ChatPromptTemplate.from_template("Explique {topico} em uma frase.")
    | ChatOpenAI(model="gpt-4o-mini", temperature=0)
    | StrOutputParser()
)

print(chain.invoke({"topico": "backpropagation"}))
```

O operador `|` é o coração do LCEL. Cada componente é um **Runnable** (`.invoke`, `.stream`, `.batch`)

LCEL Sequencial: Multi-Stage Pipelines

```
# Pipeline: Texto Resumo Sentimento
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

# Stage 1: Summary
chain_resumo = (
    ChatPromptTemplate.from_template("Resuma em 2 frases: {texto}")
    | llm | StrOutputParser()
)

# Stage 2: Sentiment
chain_sentimento = (
    ChatPromptTemplate.from_template("Classifique o sentimento: {resumo}")
    | llm | StrOutputParser()
)

# Sequential Pipeline
pipeline = {"resumo": chain_resumo} | chain_sentimento

res = pipeline.invoke({"texto": "O produto chegou com defeito..."})
print(res) # "negativo"
```

Encadeamento Logico

O output da primeira chain é injetado automaticamente como input para a próxima através do mapeamento de chaves no dicionário.

Dica: Passthrough

Use `RunnablePassthrough` para manter dados originais vivos durante todo o pipeline se precisar deles no estágio final.

Texto Sentimento

RunnableBranch: Roteamento Lógico

```
# Definição do Roteamento com RunnableBranch
from langchain_core.runnables import RunnableBranch, RunnableLambda
from langchain_openai import ChatOpenAI

# Classificador de roteamento
def eh_tecnica(entrada: dict) -> bool:
    palavras = ["erro", "bug", "api", "código"]
    return any(p in entrada["pergunta"].lower() for p in palavras)

# Estrutura de Decisão
router = RunnableBranch(
    (RunnableLambda(eh_tecnica), chain_tecnica),
    chain_simples # fallback default
)

# Exemplos de Execução
print(router.invoke({"pergunta": "como resolver bug api?"}))
# → Direciona para chain_tecnica
print(router.invoke({"pergunta": "o que é ML?"}))
# → Direciona para chain_simples
```

Objetivo

O **RunnableBranch** permite criar caminhos condicionais em uma chain. Ele avalia uma série de condições e executa a primeira que retornar verdadeiro.

Dica de Fluxo

- Use **RunnableLambda** para funções de verificação customizadas.
- Sempre defina uma chain **default** ao final para garantir que o fluxo não quebre.
- Perfeito para Chatbots com diferentes personas.

Combinando Chains



Pipeline: classificação → geração → formatação

```
# Setup de Imports e LLM
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Definição das Etapas
chain_classificar = (ChatPromptTemplate.from_template("Classifique...") | llm | StrOutputParser())
chain_responder = (ChatPromptTemplate.from_template("Tipo: {tipo}...") | llm | StrOutputParser())
formatar = RunnableLambda(lambda x: f"[RESPOSTA FINAL]\n{x.strip().upper()}")

# Orquestração do Pipeline
pipeline = (
    RunnablePassthrough.assign(tipo=chain_classificar)
    | RunnablePassthrough.assign(resposta=chain_responder)
    | RunnableLambda(lambda x: x["resposta"])
    | formatar
)
```

Exemplo de Execução:

```
print(pipeline.invoke({"pergunta": "Meu boleto venceu, o que faço?"}))
```

Fluxo de Dados: Tipos de topologia de pipeline

Topologia	Quando usar	Componente
Sequencial	Cada etapa depende da anterior	<code>chain1 chain2</code>
Paralela	Etapas independentes, mesmo input	<code>RunnableParallel</code>
Condicional	Rota depende do conteúdo	<code>RunnableBranch</code>
Com memória	Acumula contexto entre turnos	<code>RunnableWithMessageHistory</code>

Decisão Baseada no LLM

Classificador de intenção como roteador — comportamento adaptativo

```
# Setup e Classificador
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

classificador = (ChatPromptTemplate.from_template( """Classifique a pergunta: técnica | comercial | suporte | outro""" ) | llm |
StrOutputParser())

# Lógica de Roteamento
chains = {"tecnica": chain_tecnica, "comercial": chain_comercial, "suporte": chain_suporte}

def rotear(entrada: dict) -> str:
    intencao = classificador.invoke(entrada)
    chain = chains.get(intencao, chain_generica)
    return chain.invoke(entrada)

pipeline_adaptativo = RunnableLambda(rotear)
```

 **Cuidado:** O LLM classificador pode errar a categoria. Sempre defina um default robusto e monitore a distribuição de rotas em produção.

Controles de Robustez

Produção significa: falhas de API, respostas inválidas, timeouts, picos de tráfego

Tratamento de Erros

Tabela de Erros comuns em pipelines com LLM e como tratar:

Tipo de Erro	Causa	Tratamento
<code>openai.RateLimitError</code>	Quota de API excedida	Retry com backoff exponencial
<code>openai.APITimeoutError</code>	Timeout na chamada	Retry + timeout maior
<code>OutputParserException</code>	LLM não gerou JSON válido	Re-prompt ou fallback
<code>ValidationError</code> (Pydantic)	Output fora do schema	Retry com instrução de correção

Estratégias

Estratégia de Fallback Nativa (.with_fallbacks)

```
# Configuração de redundância entre modelos
llm_principal = ChatOpenAI(model="gpt-4o") # Primário (Capaz)
llm_fallback1 = ChatOpenAI(model="gpt-4o-mini") # Secundário (Barato)
llm_fallback2 = ChatGroq(model="llama3-8b") # Terciário (Provedor Diferente)

# Cadeia de falhas: tenta o próximo se o anterior falhar
llm_final = llm_principal.with_fallbacks([llm_fallback1, llm_fallback2])

chain = prompt | llm_final | StrOutputParser()
```

Retry Automático

```
llm.with_retry(stop_after_attempt=3)
```

Tenta novamente a mesma chamada em caso de erro transiente na API.

Fluxo de Execução

GPT-4o → GPT-4o-mini → Llama 3 (Groq)

Garante que a pipeline não quebre se o provedor principal estiver instável ou atingir rate limits.

Logging

Funcionalidades Principais

- Registrar entradas e saídas
- Rastrear a execução do pipeline



Essencial para produção

LangSmith (Plataforma Oficial)

Configuração simples via variáveis de ambiente:

```
LANGCHAIN_TRACING_V2=true  
LANGCHAIN_API_KEY=[SUA CHAVE]
```

Registra automaticamente todas as execuções com UI visual — **sem alterar uma linha do pipeline.**

Debugging

1. verbose=True — Imprime prompt e resposta de cada chamada

Exemplo de Implementação

Habilita o log detalhado por componente

```
pythonllm_verbose = ChatOpenAI(  
model="gpt-4o-mini",  
verbose=True  
)
```

Debugging

2. Testar etapas isoladamente — "unit test" do pipeline

Snippets de Teste Unitário

```
# Testar só o pré-processamento:
print(etapa_pre.invoke({"pergunta": " TESTE "}))

# Testar só o prompt (sem chamar o LLM):
print(prompt.invoke({"pergunta": "o que é IA?", "contexto": "..."}))

# Testar só o parser:
from langchain_core.messages import AIMessage
print(parser.invoke(AIMessage(content='{"chave": "valor"}')))
```

Boas Práticas

Modularização e Reutilização

Princípios de modularização para pipelines com LLM:

Responsabilidade Única

Cada chain faz exatamente uma coisa.



Sinal de violação:

Chain com mais de 6 etapas

Testabilidade Isolada

Cada etapa pode ser testada com um único `.invoke()`.



Sinal de violação:

Não consigo testar sem rodar o pipeline inteiro

Configurabilidade

Parâmetros como modelo e temperatura são argumentos.



Sinal de violação:

Modelo hardcoded no meio do código

Nomeação Descritiva

O nome da chain diz o que ela faz, não como.



Sinal de violação:

chain1, c, temp_chain

Reusabilidade

A mesma chain aparece em mais de um pipeline.



Sinal de violação:

Código duplicado entre pipelines

Modularização e Reutilização

Estrutura de projeto recomendada:

```
src/  
├── chains/  
│   ├── pre_processamento.py  
│   ├── geracao.py  
│   └── pos_processamento.py  
├── pipelines/  
│   ├── atendimento.py  
│   └── rag.py  
└── tests/  
    ├── test_chains.py  
    └── test_pipelines.py
```

chains/

- **pre_processamento:** limpeza, normalização, validação
- **geracao:** chains de geração por domínio
- **pos_processamento:** parsers e formatadores

pipelines/

- **atendimento:** combina chains para atendimento
- **rag:** pipeline de recuperação + geração

tests/

- **test_chains:** testa cada chain isoladamente
- **test_pipelines:** testa fluxos completos

Uso Excessivo de LLMs e Pipelines

coding in 2022

JS stringUtils.js ×

```
1 function startsWithCapitalLetter(str) {  
2   return str.length > 0 && str[0] === str[0].toUpperCase();  
3 }
```

coding in 2027

JS stringUtils.js ×

```
1 import Anthropic from '@anthropic-ai/sdk';  
2  
3 const anthropic = new Anthropic({  
4   apiKey: process.env.ANTHROPIC_API_KEY,  
5 });  
6  
7 export async function startsWithCapitalLetter(str) {  
8   const prompt = `You are a capitalization expert.  
9   You have been studying capitalization for years  
10  
11   Text to analyze: "${str}"  
12  
13   Respond with ONLY 'true' or 'false'.  
14   Is the first letter of the text a capital letter?`;  
15  
16   const response = await anthropic.messages.create({  
17     model: 'claude-mythos-1-20261225',  
18     max_tokens: 500000,  
19     temperature: 0,  
20     thinking: 'ultramaxmegathink',  
21     messages: [{ role: 'user', content: prompt }],  
22   });  
23  
24   const answer = response.content[0].text.trim().toLowerCase();  
25   return answer === 'true';
```

Instabilidades do Langchain

Lembrete Final Importante

Esta aula foca no **racional das pipelines**.

Se um código quebrar no seu ambiente, o problema quase sempre é de **versão**, não de lógica.

Entendendo o racional, você será capaz de **adaptar o código** a qualquer versão futura.

Take-home Message

Pipelines & Runnables

Sequência de Runnables com entrada e saída tipadas.

✓ **Testável isoladamente**

Legado vs. LCEL

LLMChain e SequentialChain são legados.

Use LCEL com |, RunnablePassthrough e RunnableBranch

Processamento

Pré e pós-processamento definem a qualidade da entrada e o contrato da saída.

Tão importantes quanto o LLM

Robustez em Produção

Robustez não é opcional para pipelines de produção.

- `.with_retry()` e `.with_fallbacks()`
- Callbacks de logging

Modularização & Responsabilidade

Modularize pelo princípio da responsabilidade única.

Cada chain faz uma coisa, bem feita e testável.